嵌入式系统 • SPI 总线 • Flash 烧录

STM32C8T6 SPI 与 W25QXX Flash 烧录桥接工程

上位机 → STM32F103C8T6 → W25QXX
工程流程、函数使用与教学实验指导书

适用平台	STM32F103C8T6（Blue Pill / 最小系统板）
存储器件	W25Q 系列 SPI NOR Flash（1MB ~ 16MB）
上位机	Scom.exe / W25Qxx Flash 烧录工具（C#）
通信方式	UART 115200 8N1 + SPI Mode 0
文档版本	V1.0
编写日期	2026-11-02

本指导书用于课程教学、实验实训与自学。建议配合 Keil MDK 工程、Scom 上位机与真实 W25QXX 芯片同步阅读。

目 录

一、项目概述与系统架构 — 目标、数据流、设计指标

二、硬件设计与接线 — 引脚、电源、信号完整性

三、软件工程结构与启动流程 — 文件职责、初始化顺序、编译方法

四、上位机与 STM32 通信协议 — I / S / E / W / R 与原始数据流

五、W25QXX 芯片基础 — SPI 模式、命令、状态、页/扇区/块

六、函数使用说明 — UART、W25QXX、命令解析、调用示例

七、关键流程剖析 — 写入流水线、读回、擦除策略、环形缓冲

八、教学与实验安排 — 学习目标、课时、六个实验

九、学生注意事项 — 电源、时钟、接线、页边界、握手、调试

十、重点与难点总结 — 必须掌握的八个关键点

十一、常见故障排查 — 现象、原因、解决办法

十二、验证、测试与性能分析 — 手工命令、示波器、吞吐量

十三、扩展练习与进阶方向 — DMA、CRC、SFDP、QSPI

附录 — 命令速查、CubeMX 配置、编译烧录、术语表

提示 阅读建议：第一次阅读先看第一、二、四、九章，建立“数据从哪里来、到哪里去”的整体认识；动手实验时再精读第六、七章；调试失败时直接查第十一章。

一、项目概述与系统架构

1.1 项目目标

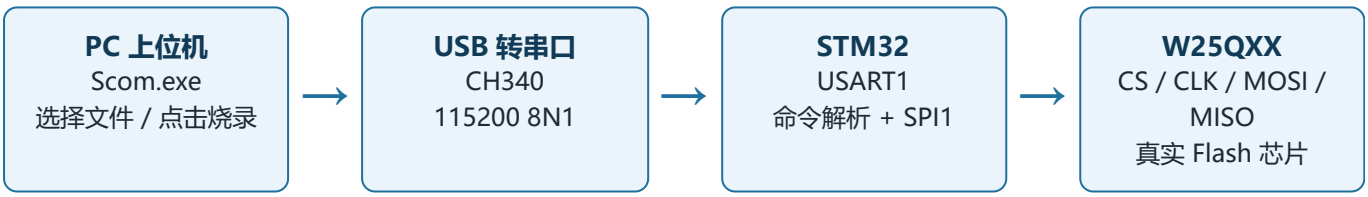
本项目把一块 STM32F103C8T6 最小系统板做成“USB 转 SPI Flash 烧录器”的核心桥接板。PC 上的 **Scom.exe** 通过 CH340 等 USB 转串口芯片，以 115200 8N1 与 STM32 的 USART1 通信；STM32 再把收到的命令翻译成 SPI 时序，对真实 W25QXX 芯片执行读 ID、擦除、写入和读回校验。

与直接使用 CH341A 等专用烧录器相比，本方案的价值在于：**学生可以完整看到从“上位机协议”到“SPI 时序”的全过程**，理解串口协议、状态机、环形缓冲、Flash 页编程、忙等待、擦除对齐等嵌入式核心知识。

1.2 系统角色划分

角色	运行位置	职责
上位机	PC，Scom.exe	选择串口和固件文件；发送 I/S/E/W/R 命令；接收原始数据；显示进度、日志与校验结果。
桥接器	STM32F103C8T6	解析 ASCII 命令；驱动 SPI1；执行 W25QXX 读、写、擦；返回 OK/ERR 与原始数据。
被烧录器件	W25QXX 芯片	SPI NOR Flash，保存固件、字库、图片或参数；按 256B 页编程、4KB 扇区擦除。
物理通道	CH340 / USB	USB 转 TTL 串口，把 PC 的 USB 数据变成 3.3V UART 电平。

1.3 数据流图



典型一次“整片烧录”的完整链路
① PC 读取固件文件 → ② ReadId 获取容量 → ③ EraseRange 按扇区擦除 → ④ Write 分页写入 → ⑤ Read 读回校验 → ⑥ 显示成功/失败。

1.4 设计指标

项目	指标 / 说明
主控	STM32F103C8T6，Cortex-M3，72MHz，64KB Flash，20KB RAM
串口	USART1，PA9=TX、PA10=RX，115200 8N1，无流控
SPI	SPI1 主机，PA4=SPI1_CS、PA5=SPI1_SCK、PA6=SPI1_MISO、PA7=SPI1_MOSI，Mode 0，软件 NSS
SPI 时钟	18MHz（72MHz / 4），兼顾速度与杜邦线可靠性

项目	指标 / 说明
Flash 容量	1MB ~ 16MB（3 字节地址）；更大容量需 4 字节地址，见扩展练习
数据完整性	上位机支持读回校验；失败时给出第一个错误地址
RAM 占用	UART 环形缓冲 2KB + 读缓冲 1KB + 页缓冲 256B，当前工程 ZI 约 5KB

重点 分清三件事：**上位机协议**（ASCII 行命令 + 原始二进制）、**STM32 桥接逻辑**（解析、缓冲、调度）、**W25QXX 芯片时序**（SPI 命令、状态轮询、页/扇区规则）。三者任何一层出错，现象都可能表现为“烧录失败”，因此要学会分层定位。

二、硬件设计与接线

2.1 器件清单

序号	器件	说明
1	STM32F103C8T6 最小系统板	带 8MHz 晶振、3.3V 稳压、SWD 下载口
2	W25QXX 模块或芯片	W25Q80 / W25Q16 / W25Q32 / W25Q64 / W25Q128 等
3	USB 转 TTL 模块	CH340、CP2102、FT232 均可，逻辑电平 3.3V
4	杜邦线、面包板	尽量短，建议不超过 15cm
5	ST-Link / DAP-Link	用于给 STM32 下载程序
6	3.3V 电源与去耦电容	Flash 电源脚附近建议 0.1μF

2.2 接线表

STM32 引脚	方向	连接对象	说明
PA9 / USART1_TX	输出	CH340 RXD	STM32 发送，接对方接收
PA10 / USART1_RX	输入	CH340 TXD	STM32 接收，接对方发送
GND	—	CH340 GND、Flash GND	**必须共地**，否则串口数据乱码
PA5 / SPI1_SCK	输出	W25QXX CLK	SPI 时钟
PA6 / SPI1_MISO	输入	W25QXX DO / IO1	Flash 输出，STM32 输入
PA7 / SPI1_MOSI	输出	W25QXX DI / IO0	STM32 输出，Flash 输入
PA4 / SPI1_CS	输出	W25QXX /CS	片选，低电平有效；由 GPIO 软件控制（软件 NSS），上电空闲必须为高
3.3V	电源	W25QXX VCC	禁止接 5V（除非模块明确带电平转换）
3.3V	电源	W25QXX /WP、/HOLD	不需要写保护/暂停时，建议上拉到 3.3V

SPI1 引脚映射（CubeMX 风格）

```
PA4     ------> SPI1_CS
PA5     ------> SPI1_SCK
PA6     ------> SPI1_MISO
PA7     ------> SPI1_MOSI
```

说明：PA4 在 CubeMX 中配置为 `GPIO_Output` 并命名为 `SPI1_CS`，由固件通过 `W25QXX_Select()` / `W25QXX_Deselect()` 手动拉低/拉高，因此 SPI1 使用 `NSS = SPI_NSS_SOFT`，而不是硬件 NSS 引脚。

2.3 硬件连接示意



2.4 硬件注意事项

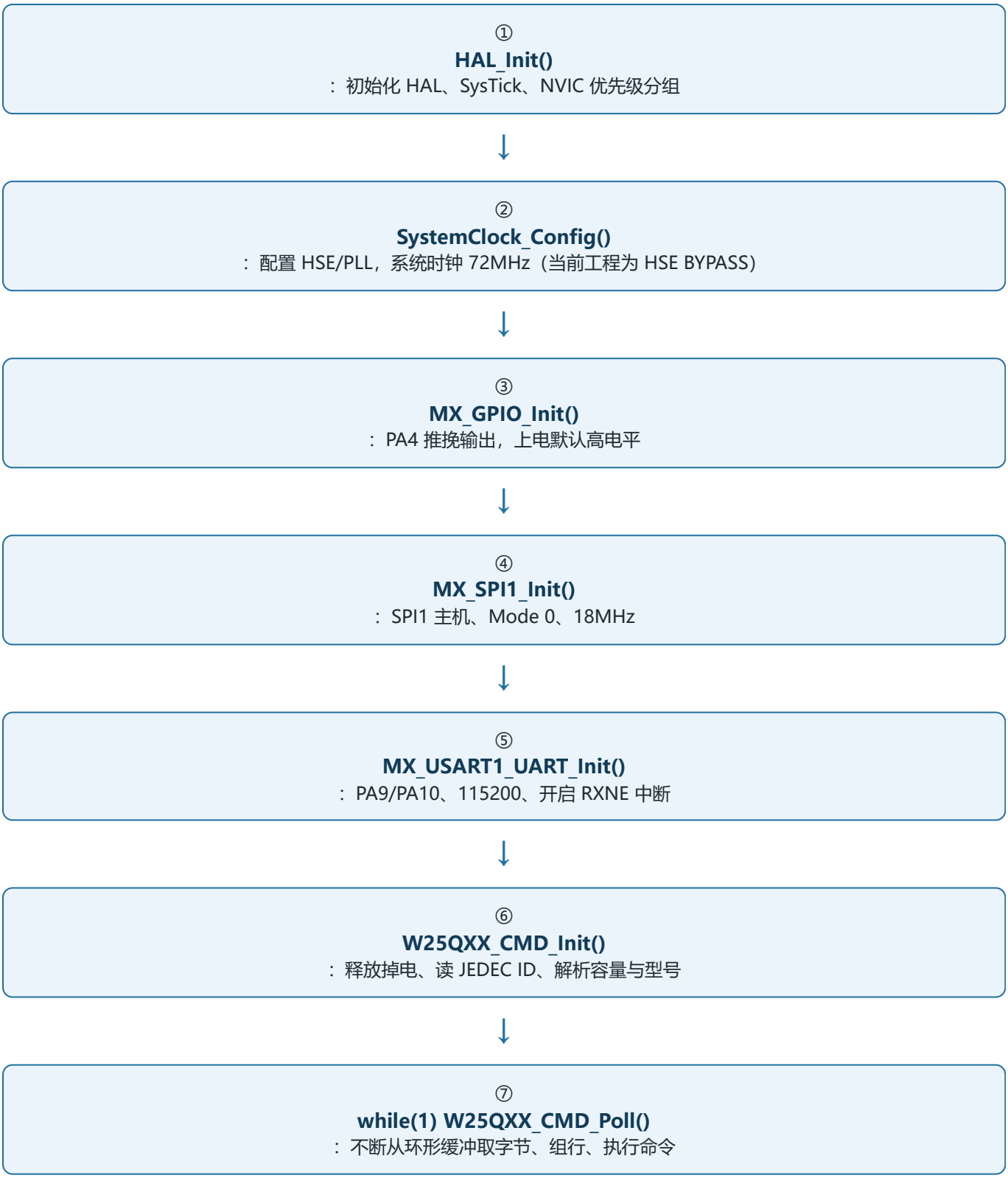
- **电平必须匹配**：W25QXX 是 3.3V 器件，SCK、MOSI、CS 高电平不能超过 VCC+0.3V。5V 单片机直连会损坏芯片。
- **共地是第一位**：PC、CH340、STM32、Flash 必须共地。只连 TX/RX 不连 GND，是“能识别串口但全是乱码”的最常见原因。
- **CS 上电必须为高**：代码在 `gpio.c` 中把 PA4 初始电平设为 `GPIO_PIN_SET`，避免上电瞬间误选中 Flash。
- **MISO/MOSI 不要接反**：Flash 的 DO 接 STM32 的 MISO（PA6），Flash 的 DI 接 STM32 的 MOSI（PA7）。
- **线缆尽量短**：SPI 18MHz 时，长杜邦线和面包板寄生参数会造成波形振铃、数据错位。优先降低 SPI 速度或缩短线缆。
- **电源去耦**：Flash 的 VCC-GND 之间靠近引脚放 0.1μF 电容，擦除瞬间电流较大，电源不稳会导致忙等待超时或数据错误。
- **不要热插拔**：擦除/写入过程中断电或拔线，可能造成数据半写；频繁热插拔还可能损坏芯片或 MCU 引脚。

三、软件工程结构与启动流程

3.1 文件职责

文件	类型	职责
`Core/Src/main.c`	应用入口	HAL 初始化、时钟配置、外设初始化、主循环轮询命令。
`Core/Src/gpio.c`	GPIO	PA4（SPI1_CS）配置为推挽输出，上电默认高电平（CS 空闲）。
`Core/Src/spi.c`	SPI	SPI1 主机、8 位、Mode 0、软件 NSS、18MHz；配置 PA5/PA6/PA7。
`Core/Src/usart.c`	UART	USART1 寄存器级初始化；RXNE 中断；2KB 环形缓冲；阻塞发送。
`Core/Inc/usart.h`	头文件	UART 对外接口声明。
`Core/Src/w25qxx.c`	Flash 驱动	JEDEC ID、读数据、页编程、扇区/块/全片擦除、忙等待、容量与型号解析。
`Core/Inc/w25qxx.h`	头文件	CS 引脚宏、页/扇区/块大小、全局信息与函数声明。
`Core/Src/w25qxx_cmd.c`	协议层	行命令解析、I/S/E/W/R 分发、原始数据收发、OK/ERR 应答。
`Core/Inc/w25qxx_cmd.h`	头文件	命令层初始化与轮询接口。
`Core/Src/stm32f1xx_it.c`	中断	SysTick 等系统异常；USART1 中断处理函数放在 usart.c 中。
`MDK-ARM/STM32C8T6_SPI.uvprojx`	Keil 工程	源文件分组、编译选项、输出 HEX。
`../src/W25QxxProgrammer.cs`	上位机	串口打开、命令封装、读 ID、擦除、写入、读回、校验。

3.2 启动流程



3.3 编译与烧录

1. 用 Keil MDK 打开 `MDK-ARM/STM32C8T6_SPI.uvprojx`。
2. 确认器件为 STM32F103C8，编译器为 ARMCC V5.06 或兼容版本。
3. 按 **F7** 编译；输出 `MDK-ARM/STM32C8T6_SPI/STM32C8T6_SPI.hex`。
4. 用 ST-Link / DAP-Link 下载 HEX，或使用 Keil 的 Download 按钮。
5. 打开 Scom.exe，选择 CH340 对应串口，进入 W25Qxx 烧录工具，选择固件文件后开始烧录。

命令行编译示例（可选）：`UV4.exe -b STM32C8T6_SPI.uvprojx -j0 -o build.log`。返回 0 表示成功。

四、上位机与 STM32 通信协议

4.1 帧格式

协议分为两种帧：**文本行命令**与**原始二进制数据**。文本行以 `\r\n` 结束，字段用空格分隔；地址和长度默认十进制，固件也兼容 `0x` 前缀的十六进制。

方向	格式	说明
PC → STM32	`命令 [参数1] [参数2]\r\n`	ASCII 文本，例如 `W 0 1024\r\n`
STM32 → PC	`OK,....\r\n` 或 `ERR,...\r\n`	文本应答，上位机按 `OK,` 前缀判断成功
PC → STM32	原始 `len` 字节	仅 W 命令握手后发送，不经过文本解析
STM32 → PC	原始 `len` 字节	仅 R 命令，紧跟在命令应答之后

> **注意** 上位机每次调用 `SendCommand()` 前会 `DiscardInBuffer()` 清空接收缓冲，然后才发送命令。因此固件**不能主动打印调试信息**，否则会被当成应答，导致 `ReadLine()` 解析错位。

4.2 命令一览

命令	PC 发送	STM32 应答	功能
I	`I\r\n`	`OK,<ID6>,<cap>,<model>\r\n`	读取 JEDEC ID、容量（字节）、型号
S	`S <start> <len>\r\n`	`OK,ERASED\r\n`	擦除覆盖 [start, start+len) 的扇区
E	`E\r\n`	`OK,ERASED\r\n`	全片擦除
W	`W <addr> <len>\r\n`	`OK,READY\r\n` → `OK,WROTE <len>\r\n`	进入写入状态，接收 len 字节并编程
R	`R <addr> <len>\r\n`	<len 字节原始数据> + `OK,READ <len>\r\n`	读回数据并发送给 PC

4.3 时序示例

(1) 读 ID

```
PC  →  "I\r\n"
MCU →  "OK,EF4017,8388608,W25Q64\r\n"
        |      |        |       └─ 型号（由厂商+容量推导）
        |      |        └──────── 容量：8,388,608 字节 = 8MB
        |      └─────────────────── JEDEC ID：EF 40 17
        └────────────────────────── 固定前缀
```

(2) 擦除范围

```
PC  →  "S 0 8388608\r\n"          （从 0 开始擦除 8MB）
MCU →  "OK,ERASED\r\n"
```

（3）写入

```
PC  →  "W 0 300\r\n"
MCU →  "OK,READY\r\n"             ← 握手成功，PC 开始发送原始数据
PC  →  300 个原始字节（非文本）
MCU →  "OK,WROTE 300\r\n"         ← 全部写完并等待 Flash 空闲后应答
```

（4）读回

```
PC  →  "R 0 16\r\n"
MCU →  16 个原始字节
MCU →  "OK,READ 16\r\n"
```

4.4 协议设计要点

- **握手隔离**：W 命令必须先回 `OK,READY`，PC 收到后才发送二进制数据；避免命令文本与数据混在一起。
- **长度驱动**：W/R 的原始数据长度完全由命令中的 `len` 决定，固件不依赖换行符判断结束。
- **OK 前缀**：上位机统一用 `StartsWith("OK,")` 判断成功，所以成功应答必须包含逗号。
- **十进制参数**：C# 使用 `CultureInfo.InvariantCulture` 输出十进制；固件同时兼容 `0x`，便于手工调试。
- **无调试输出**：任何 `printf`、启动横幅、日志都会破坏协议，调试信息应通过独立串口或调试器输出。
- **超时设计**：擦除 4KB 扇区约几十毫秒，全片擦除可达数十秒到数分钟；上位机按扇区数动态计算超时，固件也必须给足忙等待时间。

五、W25QXX 芯片基础

5.1 SPI 工作模式

W25QXX 支持 SPI Mode 0（CPOL=0，CPHA=0）和 Mode 3（CPOL=1，CPHA=1）。本工程使用 **Mode 0**：时钟空闲为低电平，数据在 SCK 上升沿采样。CubeMX 中对应 `CLKPolarity = LOW`、`CLKPhase = 1EDGE`。

参数	本工程设置
主机/从机	STM32 为主机（Master）
数据宽度	8 位
方向	全双工 2 线（SCK + MOSI + MISO）
片选	软件 NSS，GPIO PA4 手动控制
位序	MSB First
时钟	18MHz（72MHz / 4）

5.2 常用命令

命令	操作码	地址	作用
Write Enable	`0x06`	无	置位 WEL，编程/擦除前必须发送
Read Status 1	`0x05`	无	读取 BUSY、WEL 等状态
Read Data	`0x03`	24 位	普通读，支持连续地址递增
Page Program	`0x02`	24 位	页编程，单次最多 256B，不能跨页
Sector Erase	`0x20`	24 位	擦除 4KB 扇区
Block Erase 32K	`0x52`	24 位	擦除 32KB 块
Block Erase 64K	`0xD8`	24 位	擦除 64KB 块
Chip Erase	`0xC7`	无	全片擦除，耗时最长
JEDEC ID	`0x9F`	无	读厂商、类型、容量代码
Release Power-down	`0xAB`	无	唤醒芯片并读取 Device ID

5.3 状态寄存器

位	名称	含义
bit0	BUSY	1 = 芯片正在编程/擦除/写状态寄存器；此时只能读状态，不能发新命令
bit1	WEL	写使能锁存；WREN 后为 1，编程/擦除完成后自动清零
bit2	BP0	块保护位（本工程不使用写保护）

位	名称	含义
bit3	BP1	块保护位
bit4	BP2	块保护位
bit5	TB	保护区域方向
bit6	SEC	扇区保护
bit7	SRP0	状态寄存器保护

重点 编程和擦除的本质都是“把 bit 从 1 变成 0”。擦除是把整个扇区恢复为 `0xFF`，写入只能把 1 改成 0。因此**写入前必须先擦除**；对同一地址重复写而不擦除，结果只会是原值与新值的按位与。

5.4 存储组织结构

单位	大小	说明
Page（页）	256 字节	页编程的最小单位；一次 Page Program 不能跨越 256 字节边界
Sector（扇区）	4KB	最小擦除单位；擦除后整扇区变为 0xFF
Block（块）	32KB / 64KB	大块擦除，提高整片擦除效率
地址	24 位	3 字节地址，最大寻址 16MB；更大容量需 4 字节地址模式

5.5 容量推导

JEDEC ID 的低字节是容量代码 `capcode`，容量（字节）= `1 << capcode`。型号（Mbit）= 容量字节 / 128KB。常见对应关系如下：

capcode	容量	常见型号	说明
0x14	1MB	W25Q80	8Mbit
0x15	2MB	W25Q16	16Mbit
0x16	4MB	W25Q32	32Mbit
0x17	8MB	W25Q64	64Mbit，实验最常用
0x18	16MB	W25Q128	128Mbit，3 字节地址上限
0x19	32MB	W25Q256	需 4 字节地址，本工程默认不覆盖

5.6 擦除与写入的时间量级

操作	典型时间	最大值（参考）	本工程超时
Page Program 256B	约 0.7ms	约 3ms	2s
Sector Erase 4KB	约 45ms	约 400ms	3s
Block Erase 32KB	约 120ms	约 1.6s	5s

操作	典型时间	最大值（参考）	本工程超时
Block Erase 64KB	约 150ms	约 2s	10s
Chip Erase	20s～40s	100s～200s	300s

具体数值以所用型号数据手册为准。教学时重点建立“擦除远慢于写入，全片擦除最慢”的时间直觉。

六、函数使用说明

6.1 UART 模块（usart.c / usart.h）

void MX_USART1_UART_Init(void)

初始化 USART1：使能 GPIOA 和 USART1 时钟，PA9 配置为复用推挽输出，PA10 配置为浮空输入，计算 BRR，开启接收中断，设置 NVIC 优先级。

```
MX_USART1_UART_Init();   // 在 main() 中外设初始化阶段调用一次
// 之后可通过 UART_SendString() 发送文本
```

void UART_Send(const uint8_t *data, uint16_t len)

阻塞式发送任意二进制数据。逐字节等待 TXE 置位后写 DR，最后等待 TC 置位，确保数据真正移出移位寄存器。

参数	说明
`data`	待发送缓冲区首地址，可为二进制数据
`len`	字节数，最大 65535

```
uint8_t buf[3] = {0x01, 0x02, 0x03};
UART_Send(buf, sizeof(buf));
```

void UART_SendString(const char *text)

发送以 `\0` 结尾的字符串，内部调用 `UART_Send`。用于发送 `OK,...` / `ERR,...` 应答。

```
UART_SendString("OK,READY\r\n");
```

int UART_RxPop(uint8_t *byte)

从环形缓冲取出一个字节。**非阻塞**：有数据返回 1，无数据返回 0。

```
uint8_t ch;
while (UART_RxPop(&ch)) {
    // 处理一个字节
}
```

int UART_RxWait(uint8_t *byte, uint32_t timeout_ms)

等待一个字节，超时返回 0，成功返回 1。用于 W 命令接收原始数据和行解析中的等待。

```
uint8_t ch;
if (UART_RxWait(&ch, 2000) == 0) {
    // 2 秒内没有收到数据，超时处理
}
```

void UART_FlushRx(void)

丢弃环形缓冲中尚未处理的字节。调试时可用来清空残留数据；正常协议流程不建议随意调用，以免丢掉命令。

void USART1_IRQHandler(void)

USART1 中断服务函数，由硬件自动调用。读取 SR 判断 RXNE/ORE/NE/FE/PE，读取 DR 清除标志，收到有效字节时压入环形缓冲。**应用层不要直接调用它。**

> **提示** 环形缓冲采用“单生产者/单消费者”模型：中断只写 `rx_head`，主循环只写 `rx_tail`，二者都是 16 位对齐访问，在 Cortex-M3 上天然原子，不需要关中断。缓冲区大小 2048 是 2 的幂，取模用位与 `& (SIZE-1)` 实现，效率高。

6.2 W25QXX 驱动（w25qxx.c / w25qxx.h）

void W25QXX_Init(void)

拉高 CS，发送 `0xAB` 释放掉电，最多重试 3 次读取 JEDEC ID；根据厂商和容量代码生成型号字符串，更新全局变量。

全局变量	含义
`g_w25qxx_jedec_id`	24 位 JEDEC ID，例如 `0xEF4017`
`g_w25qxx_capacity`	容量（字节），例如 8388608
`g_w25qxx_model`	型号字符串，例如 `"W25Q64"`

```
W25QXX_Init();
printf("ID=%06lX, cap=%lu, model=%s\r\n",
       g_w25qxx_jedec_id, g_w25qxx_capacity, g_w25qxx_model);
```

uint32_t W25QXX_ReadJedecId(void)

发送 `0x9F`，读取 3 字节并组合为 `厂商<<16 | 类型<<8 | 容量代码`。返回值 `0xFFFFFF` 通常表示 MISO 无响应、芯片未供电或 CS/接线错误。

int W25QXX_ReadData(uint32_t addr, uint8_t *buffer, uint32_t length)

从指定地址连续读取 `length` 字节。内部按 4096 字节分块，每块发送 `0x03 + 24 位地址` 后调用 HAL SPI 接收。返回 0 成功，-1 失败或越界。

```
uint8_t id[3];
if (W25QXX_ReadData(0x000000, id, sizeof(id)) == 0) {
    // id[0] 为地址 0 的内容
}
```

int W25QXX_WriteData(uint32_t addr, const uint8_t *buffer, uint32_t length)

从指定地址写入任意长度数据。函数自动按 256 字节页边界拆分：每页先发 WREN，再发 `0x02 + 地址 + 数据`，最后轮询 BUSY 直到完成。返回 0 成功，-1 失败或越界。

```
uint8_t data[300];
// 填充 data ...
if (W25QXX_WriteData(0x000000, data, sizeof(data)) == 0) {
    // 写入成功（前提：该区域已擦除）
}
```

> **注意** `W25QXX_WriteData()` 不会自动擦除。调用前必须保证目标区域已经是 `0xFF`，否则写入结果是“旧值 AND 新值”。整片烧录时上位机会先发 S 命令擦除。

int W25QXX_EraseRange(uint32_t addr, uint32_t length)

擦除覆盖 `[addr, addr+length)` 的所有扇区。函数优先使用 64KB 块擦除，其次 32KB，最后 4KB 扇区擦除，以减少命令次数。返回 0 成功，-1 失败或越界。

```
// 擦除前 64KB
if (W25QXX_EraseRange(0, 65536) == 0) {
    // 擦除完成，读回应为 0xFF
}
```

注意：只要范围与某个扇区有交集，整个扇区都会被擦除；范围外的数据可能丢失。上位机在 `EraseRange()` 中已把起止地址按 4KB 对齐。

int W25QXX_EraseChip(void)

发送 `0xC7` 执行全片擦除，忙等待最长 300 秒。返回 0 成功，-1 超时或失败。

```
if (W25QXX_EraseChip() == 0) {
    // 整片 Flash 已变为 0xFF
}
```

内部辅助函数

函数	作用
`W25QXX_Select() / Deselect()`	拉低/拉高 PA4，控制片选
`W25QXX_Transfer()`	SPI 全双工交换 1 字节
`W25QXX_SendAddress()`	发送 24 位地址（高字节在前）
`W25QXX_ReadStatus1()`	读状态寄存器 1
`W25QXX_WaitReady()`	轮询 BUSY 位直到空闲或超时
`W25QXX_WriteEnable()`	发送 0x06 并校验 WEL
`W25QXX_WakeUp()`	发送 0xAB 释放掉电模式
`W25QXX_BuildInfo()`	由 ID 推导容量和型号

6.3 命令协议层（w25qxx_cmd.c / w25qxx_cmd.h）

void W25QXX_CMD_Init(void)

复位行缓冲状态，调用 `W25QXX_Init()` 完成 Flash 上电初始化。应在 UART 初始化之后调用。

void W25QXX_CMD_Poll(void)

从 UART 环形缓冲不断取字节：遇到 `\r` 或 `\n` 就组成一行并分发命令；缓冲溢出时丢弃该行并重新同步。**必须在主循环中反复调用。**

```
while (1) {
    W25QXX_CMD_Poll();
}
```

注意：该函数内部的擦除、写入、读回是阻塞执行的。对专用烧录桥接器来说可以接受；若要同时处理其他任务，需要改成状态机或 RTOS 任务。

内部函数

函数	作用
`CMD_ParseU32()`	解析十进制/0x 十六进制无符号整数
`CMD_ReadExact()`	从环形缓冲精确读取 N 字节，带超时
`CMD_RangeValid()`	检查地址和长度是否超出容量
`CMD_HandleId()`	处理 I 命令，失败返回 ERR,NOFLASH
`CMD_HandleEraseRange()`	处理 S 命令
`CMD_HandleEraseChip()`	处理 E 命令
`CMD_HandleWrite()`	处理 W 命令：READY 握手、按页接收、编程、WROTE 应答
`CMD_HandleRead()`	处理 R 命令：分块读 Flash 并发送原始数据
`CMD_HandleLine()`	跳过前导非字母字符，识别命令并分发

6.4 在自建工程中调用

```
#include "usart.h"
#include "w25qxx.h"

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_SPI1_Init();
    MX_USART1_UART_Init();
    W25QXX_Init();

    uint8_t buf[256];
    for (int i = 0; i < 256; i++) buf[i] = (uint8_t)i;

    if (W25QXX_EraseRange(0, 4096) != 0) { Error_Handler(); }
    if (W25QXX_WriteData(0, buf, 256) != 0) { Error_Handler(); }

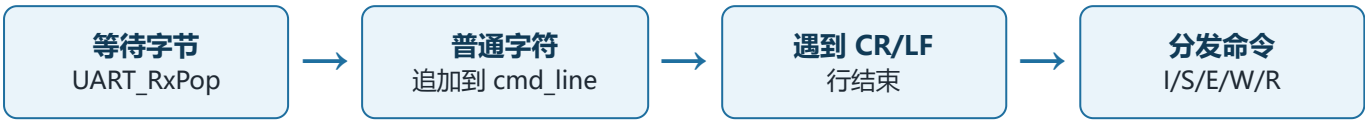
    uint8_t check[256];
    if (W25QXX_ReadData(0, check, 256) != 0) { Error_Handler(); }

    while (1) { }
}
```

七、关键流程剖析

7.1 命令解析状态机

`W25QXX_CMD_Poll()` 本质上是一个“按行收字节”的小型状态机：



- 行长度上限 `CMD_LINE_SIZE = 96`，超长则丢弃整行，防止缓冲溢出。
- 行首若混入 `0x00`、`0xFF` 等上电噪声，`CMD_HandleLine()` 会跳到第一个字母，提升首次连接成功率。
- 空行直接忽略，不产生应答。

7.2 写入流水线（W 命令）

```
PC: "W addr len\r\n"
        |
        ▼
固件: 检查地址范围 ──失败──→ "ERR,RANGE\r\n"
        |成功
        ▼
固件: "OK,READY\r\n"
        |
        ▼
PC: 连续发送 len 字节（每块 16KB，块间 sleep 1ms）
        |
        ▼
固件循环:
    chunk = min(剩余, 256 - (addr & 0xFF))
    CMD_ReadExact(page_buffer, chunk)     // 从环形缓冲取数据
    W25QXX_WriteData(addr, page_buffer, chunk)
        ├─ WriteEnable (0x06)
        ├─ PageProgram (0x02 + 地址 + 数据)
        └─ WaitReady  (轮询 BUSY)
    addr += chunk
        |
        ▼
固件: "OK,WROTE len\r\n"
```

为什么按页拆分？ W25QXX 的 Page Program 不能跨 256 字节页边界。若一次发送超过页尾的字节，芯片会在页内回卷，导致后面的数据覆盖页首，产生“看起来写了但内容错位”的隐蔽错误。

> **重点** 写入流程中的四个“必须”：必须发 WREN、必须按页拆分、必须等待 BUSY 清零、必须保证目标区域已擦除。缺任何一个都可能表现为“读回校验失败”。

7.3 读回流水线（R 命令）

```
PC: "R addr len\r\n"
        |
        ▼
固件：检查范围
        |
        ▼
循环，每次最多 1024 字节：
    W25QXX_ReadData(addr, read_buffer, chunk)   // 0x03 + 地址 + 连续读
    UART_Send(read_buffer, chunk)               // 原始二进制
    addr += chunk; 剩余 -= chunk
        |
        ▼
固件："OK,READ len\r\n"
```

上位机的 `ReadRaw()` 先按长度读满原始字节，再 `ReadLine()` 读取完成标志。因此固件必须保证“先数据、后 OK 行”的顺序，且数据长度与命令完全一致。

7.4 擦除策略

`W25QXX_EraseRange()` 按地址对齐情况自动选择擦除粒度：

条件	使用命令	目的
地址 64KB 对齐且剩余 ≥ 64KB	`0xD8` 64KB 块擦除	减少命令数量，整片擦除更快
地址 32KB 对齐且剩余 ≥ 32KB	`0x52` 32KB 块擦除	折中粒度
其他情况	`0x20` 4KB 扇区擦除	处理首尾不对齐部分

例如擦除 8MB：从地址 0 开始连续 128 次 64KB 块擦除即可；若只用 4KB 扇区需要 2048 次，命令开销和等待时间都更大。

7.5 为什么必须用中断 + 环形缓冲

这是本工程最容易被学生忽略、但最关键的工程细节之一。做一次定量估算：

项目	数值
波特率 115200，8N1，每字节 10 bit	11520 字节/秒 ≈ 11.52 字节/毫秒
一个字节的传输时间	约 86.8μs
Page Program 256B 最大耗时	约 3ms
3ms 内到达的串口数据量	约 35 字节
STM32 USART 接收数据寄存器深度	1 字节

如果采用“查询式接收 + 写一页时完全不理串口”的写法，在写页的 3ms 内就会发生**溢出（ORE）**，丢失约 34 字节，后续数据全部错位。加入 RXNE 中断后，CPU 在写 Flash 的同时由中断把字节存入 2KB 环形缓冲，35 字节远小于 2KB，因此不会丢数据。

> **提示** 如果以后把波特率提高到 460800 或 921600，或者增加更长的阻塞操作，应同步评估环形缓冲大小。经验公式：`缓冲 ≥ 最大阻塞时间(ms) × 波特率/10/1000 × 安全系数` 。

7.6 阻塞式设计的取舍

本工程为了代码清晰，擦除、写入、读回都在 `W25QXX_CMD_Poll()` 内部阻塞完成。优点是流程直观、无需状态机；缺点是烧录期间 MCU 不能响应其他业务。对“专用烧录桥”这是合理的。若要在同一 MCU 上同时做显示、采集、通信，应改为非阻塞状态机或 RTOS 任务，把 Flash 操作放到低优先级任务中。

八、教学与实验安排

8.1 学习目标

维度	目标
知识	理解 UART 帧格式、SPI Mode 0 时序、NOR Flash 页/扇区/块结构、JEDEC ID、状态寄存器、忙等待。
技能	能独立完成 CubeMX/Keil 配置；能用逻辑分析仪或示波器观察 SPI；能编写命令解析与环形缓冲；能定位“能读 ID 但写不进去”的故障。
方法	掌握“分层调试”：先电源与时钟，再串口，再 SPI，再协议，最后整片烧录。
素养	建立数据手册意识、时序意识、边界意识、校验意识和安全意识。

8.2 课时建议

课时	内容	形式
1	项目演示、系统框图、器件认识	讲授 + 实物展示
2	UART 初始化、环形缓冲、I 命令	代码精读 + 串口助手验证
3	SPI Mode 0、JEDEC ID、读数据	示波器/逻辑分析仪观察
4	扇区擦除、页编程、忙等待	实验 + 读回验证
5	上位机协议、W/R 原始数据流	抓包 + 协议分析
6	整片烧录、校验、性能测试	综合实验
7	故障注入与排查	分组排障竞赛
8	扩展任务与答辩	项目式考核

8.3 实验一：串口通信与读 ID

目标：验证电源、时钟、串口、SPI 接线与 JEDEC ID 读取。

1. 按接线表连接 CH340 与 STM32，Flash 先不接。
2. 下载固件，打开串口助手（115200 8N1，发送新行）。
3. 发送 `I`，观察是否返回 `ERR,NOFLASH`。
4. 接入 W25QXX，再次发送 `I`，记录 ID、容量、型号。
5. 对照数据手册，确认 ID 的厂商、类型、容量代码。

验收：能解释 `EF 40 17` 各字节含义，并说出对应容量。

8.4 实验二：擦除与空白读

目标：理解扇区擦除和 0xFF。

1. 发送 `S 0 4096`，等待 `OK,ERASED`。

2. 发送 `R 0 256`，用十六进制显示，确认全部为 `FF`。
3. 发送 `S 0 65536`，比较 4KB 与 64KB 擦除的耗时差异。

验收：能说明“擦除为什么以扇区为单位”“为什么不能只擦一个字节”。

8.5 实验三：页边界写入

目标：掌握 256 字节页边界。

1. 构造 300 字节递增数据，从上位机写入地址 0（先擦除）。
2. 读回 300 字节，逐字节比较。
3. 再用“单次 300 字节页编程”的错误写法做对比，观察页回卷导致的错位。

验收：能画出 0～255、256～511 两页的拆分过程，并解释页回卷。

8.6 实验四：整片烧录与校验

目标：完成真实固件文件的完整烧录流程。

1. 准备一个 32KB 以内的测试文件（可用图片或字库）。
2. 在 Scom 中连接串口，读取 ID，确认容量。
3. 勾选“校验”，开始烧录，观察擦除、写入、读回三个阶段。
4. 烧录完成后再次读回，与源文件做二进制比较。

验收：日志出现“校验通过”，并能说出 W 命令的 READY/WROTE 两次握手分别发生在什么时刻。

8.7 实验五：故障注入

目标：建立排障思维。

- 拔掉 MISO：现象是什么？
- 把 CS 一直拉低：现象是什么？
- 把波特率改成 9600 而固件仍为 115200：现象是什么？
- 写之前不擦除：读回数据为什么是“旧值 AND 新值”？
- 把 SPI 模式改成 Mode 2：读 ID 是否还正确？

验收：每组提交“现象→原因→验证方法→修复”四段式报告。

8.8 实验六：性能与稳定性

目标：定量分析瓶颈。

1. 测量擦除 8MB、写入 1MB、读回 1MB 的耗时。
2. 计算 UART 理论下限：8MB ÷ 11520 B/s ≈ 728 秒。
3. 说明为什么提高 SPI 速度对整片烧录时间影响很小。
4. 观察写入过程中是否出现丢字节，验证环形缓冲的作用。

验收：能指出“当前系统的瓶颈在 UART，不在 SPI”。

九、学生注意事项

9.1 电源与电平

- W25QXX 必须使用 **3.3V**。不要接 5V，不要用 ST-Link 的 5V 直接供 Flash。
- 所有设备必须**共地**：PC/CH340/STM32/Flash 的 GND 连在一起。
- 擦除瞬间电流较大，USB 供电不足可能导致电压跌落；建议使用带稳压的 3.3V 电源并加 0.1μF 去耦电容。

9.2 时钟配置（最容易翻车）

> **重点** 当前 `SystemClock_Config()` 使用 `RCC_HSE_BYPASS`，意思是 **OSC_IN 输入外部时钟信号**，不是晶振模式。如果你的板子是常见的 8MHz 晶振（Blue Pill），必须把 `RCC_HSE_BYPASS` 改为 `RCC_HSE_ON`，否则程序会在时钟初始化处卡死，串口完全无反应。

判断方法：用示波器看 PD0/OSC_IN 是否有 8MHz 波形；或者直接改成 `RCC_HSE_ON` 测试。若没有晶振也没有外部时钟，可临时改用 HSI + PLL（64MHz），但串口和 SPI 的分频参数要重新核对。

9.3 接线与引脚

- TX/RX 必须交叉：STM32 PA9(TX) → CH340 RXD；PA10(RX) ← CH340 TXD。
- SPI 的 MISO/MOSI 不要接反：PA6 接 Flash 的 DO，PA7 接 Flash 的 DI。
- CS 上电空闲必须为高，否则 Flash 一直处于选中状态，读 ID 可能得到 0xFFFFFF。
- 不要占用 PA13/PA14（SWDIO/SWCLK），否则无法下载和调试。
- 杜邦线尽量短；SPI 出现随机错误时，先降低 SPI 速度。

9.4 串口

- 上位机固定 **115200 8N1 无流控**，固件必须一致。
- 不要开启奇偶校验、硬件流控或额外停止位。
- 不要主动输出调试信息，否则会破坏协议。
- 读 ID 返回 `ERR,NOFLASH` 时，先查 MISO、CS、供电，再查 SPI 模式。

9.5 SPI 模式与速度

- W25QXX 用 Mode 0：`CLKPolarity = LOW`、`CLKPhase = 1EDGE`。模式选错，ID 可能偶尔能读对，但写数据一定错。
- 本工程 SPI 为 18MHz。若杜邦线较长，可降到 9MHz（预分频 8）验证稳定性。
- SPI 是“主从同时交换”：读数据时也要发 dummy 字节（0xFF）产生时钟。

9.6 Flash 操作规则

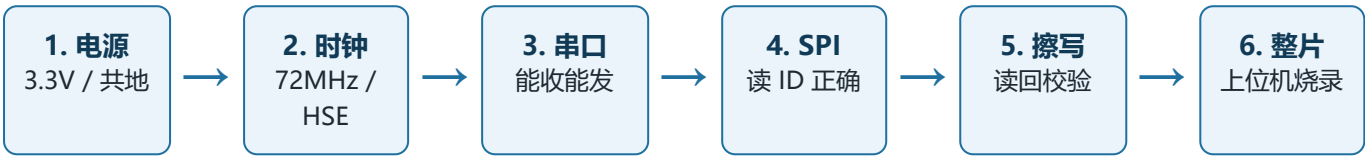
- **先擦后写**：写入只能把 1 变 0。未擦除区域写入会得到旧值 AND 新值。
- **页边界**：Page Program 单次最多 256 字节且不能跨页；跨页会回卷覆盖。
- **写使能**：每次编程/擦除前都要发 `0x06`，并确认 WEL=1。
- **忙等待**：编程/擦除后必须轮询 BUSY，直到清零才能发下一条命令。

- **擦除粒度**：最小擦除单位是 4KB 扇区，不能只擦一个字节。
- **地址范围**：写入/擦除前检查 `addr + len` 是否超过容量，避免回卷。

9.7 协议与缓冲

- W 命令必须先回 `OK,READY`，再接收原始数据；顺序不能反。
- W/R 的原始数据长度由命令参数决定，不能靠换行判断。
- 接收必须使用中断 + 环形缓冲，不能在写 Flash 时停止收串口，否则必丢字节。
- 环形缓冲下标用 16 位变量，缓冲区大小为 2 的幂，掩码取模。
- 行缓冲要限长；超长时整行丢弃，避免溢出。

9.8 调试顺序



不要一上来就整片烧录。先用 `I` 命令确认链路，再用 4KB 小范围验证擦写，最后才烧大文件。

9.9 安全与规范

- 擦除/写入过程中不要断电、不要拔线，否则可能产生半写数据。
- 拿芯片前先放电，注意 ESD；不要用手直接摸引脚。
- 不要把 Flash 的 VCC 接到 5V，也不要把 5V 信号直接灌入 MISO/MOSI。
- 实验完成后先断电再拆线，避免短路。

十、重点与难点总结

如果只能记住八件事，请记住下面这些。它们覆盖了本工程 90% 的失败场景。

序号	重点	为什么重要 / 怎么记
1	时钟与引脚	HSE BYPASS 与晶振模式不能混；TX/RX 交叉、MISO/MOSI 不反、CS 空闲高。错了后面全错。
2	SPI Mode 0	CPOL=0、CPHA=0。模式错会导致 ID 偶尔对、数据必错。
3	W 命令两次握手	`OK,READY` 表示“可以发数据了”；`OK,WROTE` 表示“写完且 Flash 空闲”。少一次都不行。
4	中断 + 环形缓冲	写一页 3ms，串口能来 35 字节，硬件只有 1 字节寄存器。没有缓冲必溢出。
5	256 字节页边界	Page Program 不能跨页；跨页会回卷覆盖。地址递增时必须重新计算本页剩余空间。
6	WREN + BUSY	每次编程/擦除前写使能，之后轮询 BUSY。跳过任何一步都会随机失败。
7	先擦后写 + 擦除粒度	写入只能 1→0；最小擦除 4KB。整片烧录先 S 再 W。
8	读回校验	写入成功不等于数据正确。必须读回逐字节比较，并报告第一个错误地址。

10.1 难点解析

难点一：为什么“能读 ID，却写不进去”？

读 ID 只验证了 SPI 基本通信，不验证 WREN、页边界、BUSY、擦除、写保护。常见原因：目标区域未擦除、WP 引脚被拉低、跨页写入、写完没等 BUSY、WriteEnable 后没检查 WEL。

难点二：为什么读回数据整体偏移/错位?

多数是接收丢字节。检查：是否用了中断接收、环形缓冲是否溢出、写页期间是否关闭了串口接收、波特率是否一致、是否在协议中混入了调试输出。

难点三：为什么偶尔成功、偶尔失败?

时序/信号完整性问题：SPI 太快、线太长、地线不良、电源去耦不足、忙等待超时太短。用示波器看 SCK 与 MISO 波形，先降速验证。

难点四：为什么全片擦除很慢?

正常现象。全片擦除是芯片内部高压操作，W25Q64 典型几十秒，W25Q128 可达数分钟。不要误判为死机；上位机已把超时设为 5 分钟。

十一、常见故障排查

现象	可能原因	排查与解决
发送 I 无任何回复	时钟未起振、串口 TX/RX 接反、波特率不对、USART 中断未使能	先量 PA9 是否有波形；检查 HSE 模式；确认 115200 8N1；确认 USART1_IRQn 已使能
回复乱码	波特率不匹配、未共地、电平不对	统一 115200；共地；确认 CH340 为 3.3V 电平
I 返回 ERR,NOFLASH	Flash 未供电、CS 未拉低/MISO 无响应、SPI 模式错	量 VCC=3.3V；查 PA4 是否拉低；查 PA6 接线；确认 Mode 0
ID 为 FFFFFF	MISO 悬空/接反、CS 一直为高、芯片未唤醒	确认 DO→PA6；CS 空闲高、选中低；发送 0xAB 唤醒
能读 ID，擦除失败	WP 引脚被拉低、写保护位、电源不足	WP/HOLD 上拉 3.3V；检查状态寄存器 BP 位；加强供电
写入后读回全 FF	没发 WREN、没擦除、地址越界、页边界错误	看逻辑分析仪是否有 0x06/0x02；先擦后写；检查地址与页拆分
写入后数据错位	跨页回卷、接收丢字节、SPI 模式错	按页拆分；确认环形缓冲；确认 Mode 0
校验随机失败	SPI 太快/线太长、地线不良、电源噪声	降到 9MHz；缩短线；加去耦；检查共地
擦除/写入超时	忙等待时间不足、芯片假死、电源跌落	延长超时；重新上电；检查 3.3V 电流能力
上位机自动连接失败	串口被主界面占用、驱动异常、MCU 未运行	关闭主界面串口；重装 CH340 驱动；复位 MCU
烧录中途停止	用户取消、串口拔出、看门狗复位	检查日志；确认没有启用 IWDG；重新连接

提示 排障时坚持“替换法 + 分段法”：换一根线、换一个串口、换一块 Flash；把链路分成 PC→CH340、CH340→STM32、STM32→Flash 三段，逐段确认。不要同时改多个变量。

十二、验证、测试与性能分析

12.1 手工命令验证

使用串口助手（如 XCOM），设置为 115200 8N1，勾选“发送新行”，逐条发送：

发送	预期回复	说明
`I`	`OK,EF4017,8388608,W25Q64`	以实际芯片为准
`S 0 4096`	`OK,ERASED`	擦除第一个 4KB 扇区
`R 0 16`	16 字节 FF + `OK,READ 16`	用 HEX 模式查看
`E`	`OK,ERASED`	全片擦除，耐心等待

W 命令涉及原始二进制，手工串口助手不方便精确发送指定长度，建议用 Scom 上位机或自行编写 Python 脚本测试。

12.2 逻辑分析仪观察点

- CS 下降沿后第一个字节应为命令码：读 ID 是 `0x9F`，读数据是 `0x03`，页编程是 `0x02`，写使能是 `0x06`。
- 读数据时观察 24 位地址是否高字节在前。
- 页编程时确认每帧数据不超过 256 字节且不跨页。
- 擦除后观察 BUSY 位从 1 变 0 的时间。
- SPI 时钟空闲应为低（Mode 0），数据在上升沿稳定。

12.3 性能估算

项目	计算 / 结果
串口有效速率	115200 / 10 = 11520 字节/秒
1MB 数据纯传输时间	1,048,576 / 11520 ≈ 91 秒
8MB 数据纯传输时间	8,388,608 / 11520 ≈ 728 秒 ≈ 12 分钟
SPI 18MHz 理论速率	约 2.25MB/s，远高于串口
结论	整片烧录瓶颈在 UART，提高 SPI 速度对总时间影响很小

若需要更快，应优先提高串口波特率（如 460800/921600），并同步增大环形缓冲、缩短上位机块间延时；而不是一味提高 SPI 时钟。

12.4 测试矩阵建议

维度	建议覆盖
芯片	W25Q16 / W25Q32 / W25Q64 / W25Q128（至少两种）
地址	0、页边界 0x100、扇区边界 0x1000、块边界 0x8000/0x10000、末尾
长度	1B、255B、256B、257B、4KB、64KB、整片

维度	建议覆盖
内容	全 0x00、全 0xFF、0x55/0xAA、递增、随机
环境	常温、不同 USB 口、不同线长、不同电源

十三、扩展练习与进阶方向

13.1 基础扩展

- **增加 LED 指示**：擦除、写入、校验时点亮不同 LED，便于观察状态。
- **增加进度应答**：每写完 4KB 返回一行进度（需修改上位机协议）。
- **增加芯片唯一 ID 读取**：使用 `0x4B` 命令读取 64 位 Unique ID。
- **增加写保护检测**：读取状态寄存器 BP/TB 位，提示用户解除保护。

13.2 性能扩展

- **UART DMA**：用 DMA 接收/发送替代中断逐字节，降低 CPU 占用，支持更高波特率。
- **SPI DMA**：读写大块数据时使用 DMA，提高吞吐并减少阻塞。
- **更高波特率**：460800/921600，需重新评估环形缓冲和上位机超时。
- **Quad SPI**：使用 IO0～IO3 四线模式，大幅提高读速度（需要芯片和引脚支持）。

13.3 功能扩展

- **SFDP 解析**：读取 `0x5A` 的 SFDP 表，自动识别容量、页大小、擦除粒度，兼容更多厂商。
- **4 字节地址**：支持 W25Q256 及以上容量，使用 `0x13` / `0x12` 或进入 4-byte 地址模式。
- **CRC 校验**：STM32 计算 CRC32，与上位机比对，减少读回数据量。
- **多片选择**：增加多路 CS，支持一拖多烧录。
- **脱机烧录**：STM32 从自身 Flash 或 SD 卡读取固件，按键触发，不依赖 PC。
- **读写文件系统**：在 Flash 上实现 LittleFS/FatFS，用于数据记录。

13.4 工程化扩展

- **非阻塞状态机**：把擦除/写入拆成状态机，主循环同时处理串口、按键、显示。
- **RTOS**：用 FreeRTOS 创建 Flash 任务、协议任务、监控任务。
- **看门狗**：在长擦除期间合理喂狗，避免误复位。
- **固件升级**：让 STM32 通过 UART 接收自身固件并写入内部 Flash，实现 IAP。

附录

附录 A：命令速查

命令	作用
`I`	读 JEDEC ID、容量、型号
`S <start> <len>`	擦除范围覆盖的扇区
`E`	全片擦除
`W <addr> <len>`	写入，先回 READY，再收 len 字节
`R <addr> <len>`	读回 len 字节，再回 OK,READ

附录 B：CubeMX / 外设配置对照

项目	配置
RCC HSE	当前工程为 BYPASS；晶振板请改 RCC_HSE_ON，PLL ×9 → 72MHz
SPI1	Master、Full-Duplex、8bit、Mode 0、NSS Soft、Prescaler 4（18MHz）、MSB First
USART1	115200、8bit、1 stop、No parity、No flow control、RX 中断
GPIO PA4 / SPI1_CS	Output Push-Pull、初始 High、Low speed；GPIO Label 设为 SPI1_CS（软件 NSS）
GPIO PA9	Alternate Function Push-Pull、High speed
GPIO PA10	Input Floating
NVIC	USART1_IRQn 使能，优先级建议高于 SysTick

附录 C：关键宏与常量

名称	值 / 含义
`W25QXX_CS_PORT / PIN`	GPIOA / GPIO_PIN_4
`W25QXX_PAGE_SIZE`	256 字节
`W25QXX_SECTOR_SIZE`	4096 字节
`W25QXX_BLOCK32_SIZE`	32768 字节
`W25QXX_BLOCK64_SIZE`	65536 字节
`UART_RX_BUFFER_SIZE`	2048 字节
`CMD_LINE_SIZE`	96 字节
`CMD_READ_BUFFER`	1024 字节

附录 D：编译与烧录步骤

1. Keil 打开 `MDK-ARM/STM32C8T6_SPI.uvprojx`。
2. F7 编译，确认 0 Error、0 Warning。
3. 确认 `STM32C8T6_SPI.hex` 已更新。
4. ST-Link/DAP-Link 连接 SWD：3.3V、GND、SWDIO、SWCLK。
5. Keil Download 或 STM32CubeProgrammer 烧录。
6. 复位，打开 Scom，选择串口，读取 Flash ID。

附录 E：术语表

术语	解释
JEDEC ID	Flash 的厂商、类型、容量标识，通常用 0x9F 读取
Page	页，256 字节，页编程单位
Sector	扇区，4KB，最小擦除单位
Block	块，32KB 或 64KB，大块擦除单位
WEL	Write Enable Latch，写使能锁存位
BUSY	芯片忙标志，1 表示正在编程/擦除
CPOL / CPHA	SPI 时钟极性与相位，决定采样边沿
环形缓冲	生产者/消费者模型，用于中断与主循环之间传递数据
回卷	页编程超过页尾时地址在页内循环，覆盖页首数据

> **提示** 建议学生每人准备一份“实验记录表”：记录芯片型号、JEDEC ID、擦除耗时、写入耗时、校验结果、遇到的问题与解决办法。工程能力的提升，来自对失败现象的持续记录和复盘。